

03_ENGINE.md

SpeechStudio Engine Specification

Version: 1.0

Status: Approved

1. Purpose

The Engine is the execution layer of SpeechStudio.

It is responsible for coordinating speech generation using Higgs Audio V3.

The Engine hides all Hugging Face and Transformers details from the user interface.

The UI communicates only with the Engine.

2. Engine Responsibilities

The Engine shall

manage the model lifecycle

manage tokenizer lifecycle

manage voice cloning

assemble prompts

validate generation requests

generate speech

post process generated audio

save outputs

store history

return generation results

3. Internal Modules

The Engine consists of the following modules.

ModelManager

PromptBuilder

VoiceManager

GenerationManager

AudioManager

HistoryManager

SettingsManager

Logger

Each module has a single responsibility.

4. Data Flow

UI

↓

Engine

↓

PromptBuilder

↓

GenerationManager

↓

ModelManager

↓

Higgs Audio V3

↓

AudioManager

↓

HistoryManager

↓

UI

5. ModelManager

Responsibilities

Load tokenizer

Load model

Unload model

Warmup model

Report model status

Report CUDA information

The ModelManager owns the only model instance.

6. PromptBuilder

Responsible for constructing the final prompt.

Receives

User text

Emotion

Style

Prosody

SFX

Voice reference

Produces

Validated Higgs prompt

Prompt preview

Validation warnings

PromptBuilder is the only component allowed to assemble prompt tokens.

7. VoiceManager

Responsible for

Loading voice profiles

Loading reference audio

Loading reference transcript

Voice validation

Voice import

Voice export

Voice metadata

VoiceManager never performs speech generation.

8. GenerationManager

Responsible for

Preparing generation request

Calling generate_speech()

Receiving generated waveform

Returning generation metadata

The GenerationManager owns generation parameters.

9. Supported Generation Parameters

The Engine shall support

temperature

top_p

top_k

max_new_tokens

seed

reference_audio

reference_sample_rate

reference_text

append_silence

normalize_output

output_format

Only officially supported parameters shall be exposed.

10. AudioManager

Responsible for

Playback

Saving WAV

Appending silence

Volume normalization

Waveform generation

Audio metadata

The AudioManager never communicates directly with the model.

11. HistoryManager

Stores

Generated prompt

Generation parameters

Voice profile

Output file

Timestamp

Generation duration

Realtime factor

History entries must be reproducible.

12. SettingsManager

Stores

Application settings

Generation defaults

Voice defaults

Window layout

Audio preferences

Settings are persisted as JSON.

13. Validation Pipeline

Before generation the Engine validates

Model loaded

Tokenizer loaded

Prompt valid

Voice profile valid

Reference audio valid

Generation parameters valid

Output directory writable

Generation is rejected if validation fails.

14. Generation Pipeline

Receive Request

↓

Validate

↓

Build Prompt

↓

Prepare Parameters

↓

Call `generate_speech()`

↓

Receive Audio

↓

Append Silence

↓

Normalize (optional)

↓

Save WAV

↓

Store History

↓

Return Result

15. Engine Events

The Engine shall publish events.

GenerationStarted

GenerationFinished

GenerationFailed

PlaybackStarted

PlaybackFinished

VoiceChanged

ModelLoaded

ModelUnloaded

HistoryUpdated

UI components subscribe to these events.

16. Worker Threads

Generation executes on a worker thread.

Audio playback executes independently.

The UI thread must never wait for generation.

17. Error Handling

Structured errors only.

Examples

ModelNotLoaded

TokenizerMissing

InvalidVoice

InvalidPrompt

GenerationFailed

CudaUnavailable

OutputWriteFailed

Errors contain

Type

Message

Suggested action

18. Logging

The Engine logs

Generation requests

Generation duration

Sampling parameters

Voice profile

Errors

Warnings

Performance data

Logs are written to

logs/

19. Performance

The Engine shall

Reuse the loaded model

Reuse the tokenizer

Reuse cached voice references

Avoid unnecessary allocations

Release temporary tensors

Keep GPU utilization high

20. Memory Ownership

ModelManager owns

Model

Tokenizer

VoiceManager owns

Reference audio

Reference transcript

GenerationManager owns

Generation request

Generation result

AudioManager owns

Playback buffers

Saved audio

Ownership shall never overlap.

21. API Between UI and Engine

The UI never communicates directly with Transformers.

The UI interacts only with Engine services.

Typical workflow

Generate Button

↓

Engine.generate()

↓

Result Object

↓

UI Update

22. Result Object

Every generation returns

Success

Generated audio path

Output duration

Generation duration

Realtime factor

Sampling settings

Voice profile

Warnings

Errors

This object is the only output returned to the UI.

23. Engine Constraints

The Engine shall never

Access UI widgets

Show dialogs

Modify UI state

Write outside the project directory

Load multiple model instances

Bypass PromptBuilder

24. Integration Points

The Engine integrates with

UI

Bootstrap

Voice Library

History

Settings

Prompt Builder

Audio Player

No other direct integrations are permitted.

25. Success Criteria

The Engine is considered successful if

the model is loaded exactly once,

all requests pass through validation,

speech generation is completed,
history is updated,
the result is returned to the UI,
and the interface remains responsive throughout the process.